

DEVOPS MASTER COURSE

Volume 6A

AWS Cloud Engineering Foundations

Chapters 1-10

Beginner to Production-Ready DevOps Engineer

Concepts • Architecture • Security • CLI • Labs • Troubleshooting • Interviews

2026 Edition

About This Book

Volume 6A is the foundation component of the AWS Cloud Engineering volume in the DevOps Master Course series. It is designed to move a learner from cloud fundamentals to the point where they can deploy, secure, scale, network, and financially govern a small production-style AWS workload.

Learning principle: Every chapter explains why the service or design exists before showing how to use it. The labs deliberately emphasize repeatability, least privilege, validation, and cleanup.

Scope and Safety

- Use a dedicated learning account wherever possible.
- Enable billing alerts before launching resources.
- Do not use the root user for daily administration.
- Never publish access keys, private keys, passwords, or account identifiers.
- Prices, quotas, Free Tier offers, and available features can change. Verify current details in the official AWS documentation and pricing pages before production use.

How to Use the Labs

1. Read the architecture and security sections before opening the console.
2. Perform the task manually once to understand the resource model.
3. Repeat it with the AWS CLI where a CLI exercise is provided.
4. Validate identity, Region, resource tags, security rules, health, logs, and cost exposure.
5. Delete chargeable resources at the end of the lab unless the next lab depends on them.

Table of Contents

Chapter 1 Cloud Computing and Amazon Web Services

Chapter 2 AWS Global Infrastructure

Chapter 3 AWS Account Setup, Free Usage Controls, and Root Security

Chapter 4 AWS Management Interfaces

Chapter 5 Identity and Access Management Deep Dive

Chapter 6 Amazon EC2 Compute Engineering

Chapter 7 Amazon Machine Images and Elastic Block Store

Chapter 8 Elastic Load Balancing and EC2 Auto Scaling

Chapter 9 Amazon VPC Networking Foundations

Chapter 10 AWS Cost Management, Governance, and Foundation Project

Appendices Command Reference, Production Readiness Checklist, Glossary, and Sources

A field-based table of contents can also be generated from the document heading styles in Word or LibreOffice.

CHAPTER 1

Cloud Computing and Amazon Web Services

Why cloud operating models changed modern infrastructure engineering

Learning Objectives

- Define cloud computing using operational characteristics rather than marketing language.
- Compare on-premises infrastructure with cloud infrastructure.
- Distinguish IaaS, PaaS, SaaS, public, private, and hybrid cloud models.
- Explain shared responsibility, elasticity, resilience, and consumption-based cost.
- Connect AWS services to the DevOps lifecycle.

1.1 Why Cloud Computing Exists

Traditional infrastructure required organizations to predict future demand, purchase servers months in advance, prepare data-center space, install operating systems, and maintain excess capacity for peak periods. Forecast errors produced one of two outcomes: insufficient capacity during demand spikes or expensive idle equipment during normal operation.

Cloud computing changes the operating model. Compute, storage, networking, databases, security controls, and managed platforms can be requested through APIs. Infrastructure therefore becomes programmable. The important shift is not simply that servers are hosted elsewhere; it is that infrastructure can be provisioned, measured, automated, replaced, and scaled through software.

DevOps connection: Cloud APIs make infrastructure automation possible. CI/CD pipelines, Infrastructure as Code, policy automation, ephemeral test environments, and automated recovery all depend on programmable infrastructure.

1.2 Essential Characteristics

Characteristic	Operational meaning	DevOps impact
On-demand self-service	Teams request capacity without waiting for manual hardware procurement.	Faster environments and shorter lead time.
Broad network access	Services are available through standard network mechanisms and APIs.	Remote operations and globally distributed delivery.
Resource pooling	Provider capacity is shared while customers remain logically isolated.	Efficient utilization without managing physical hosts.

Characteristic	Operational meaning	DevOps impact
Rapid elasticity	Capacity can increase or decrease as demand changes.	Automatic scaling and safer release events.
Measured service	Usage is metered and associated with cost.	FinOps, showback, chargeback, and cost-aware design.

1.3 Service Models

Model	Customer manages	Provider manages	Typical example
IaaS	OS, runtime, application, data, most configuration	Facilities, hardware, virtualization	EC2 virtual machines
PaaS	Application code and data	OS, runtime, scaling platform, much of operations	Managed application platforms
SaaS	Configuration and business usage	Complete application stack	Hosted collaboration or CRM software

1.4 Deployment Models

- Public cloud: provider-operated infrastructure offered to many customers with logical isolation.
- Private cloud: cloud-like automation and self-service dedicated to one organization.
- Hybrid cloud: coordinated use of on-premises and public-cloud environments.
- Multi-cloud: deliberate use of services from more than one public-cloud provider.

1.5 AWS as a Cloud Platform

AWS exposes infrastructure and managed services through the Management Console, CLI, SDKs, and APIs. The platform spans compute, storage, networking, databases, containers, serverless computing, security, observability, analytics, and developer tooling. A DevOps engineer must understand both individual services and the operational relationships between them.

1.6 Shared Responsibility Model

AWS secures the infrastructure that runs AWS services. Customers secure their workloads, identities, data, configurations, guest operating systems where applicable, and application behavior. The exact boundary changes by service type: the customer manages more for EC2 than for a highly managed serverless service.

Common failure: A managed service does not remove customer responsibility. Public S3 access, excessive IAM permissions, weak application authentication, and unpatched EC2 operating systems remain customer-controlled risks.

1.7 Cloud Economics

Cloud replaces large up-front capital expenditure with consumption-oriented operational expenditure, but it does not guarantee lower cost. Elasticity lowers cost only when resources are right-sized, stopped, scaled in, deleted, or purchased under suitable pricing models. Unowned resources and permanent overprovisioning simply move waste into a monthly bill.

1.8 Production Design Principles

- Design for failure and automate recovery.
- Prefer repeatable deployment over manual configuration.
- Deploy across failure boundaries when availability matters.
- Use least privilege and short-lived credentials.
- Collect metrics, logs, traces, and audit records.
- Tag resources and assign cost ownership.
- Treat backups as recoverability systems and test restoration.

Chapter Summary

- Cloud computing is an API-driven operating model, not merely remote hosting.
- Elasticity, automation, and measured usage create both engineering opportunities and governance obligations.
- AWS supports the complete DevOps lifecycle, but customers remain responsible for secure workload configuration.

Interview Questions

1. How is cloud computing different from traditional hosting?
2. Explain elasticity versus scalability.
3. Compare IaaS, PaaS, and SaaS using operational responsibility.
4. What does shared responsibility mean for an EC2 workload?
5. Why can a cloud migration increase rather than reduce cost?
6. How does cloud infrastructure enable DevOps practices?

Hands-on Lab 1 - Build a Cloud Service Map

Objective: Map an ordinary three-tier application to AWS service categories without deploying resources.

Tasks

1. Draw a client, web tier, application tier, database tier, object storage, DNS, monitoring, identity, and backup path.
2. For each component, identify one possible AWS service and state whether it is compute, storage, networking, database, security, or operations.
3. Label the parts managed by AWS and those managed by the customer.
4. Identify two failure scenarios and describe how redundancy or recovery could reduce impact.

5. List three cost drivers and how each could be controlled.

Validation

- Every application component has an owner and a service category.
- The design includes security, observability, backup, and cost—not only compute.

CHAPTER 2

AWS Global Infrastructure

Regions, Availability Zones, edge infrastructure, resilience, and data placement

Learning Objectives

- Explain Regions, Availability Zones, and edge locations.
- Choose a Region using latency, compliance, service availability, resilience, and cost.
- Differentiate high availability from disaster recovery.
- Design a basic multi-AZ application.
- Identify the operational consequences of Region-scoped and global services.

2.1 Regions

An AWS Region is a separate geographic area containing multiple isolated Availability Zones. Many resources—including VPCs, EC2 instances, EBS volumes, and load balancers—are created in a selected Region. Region choice is therefore an architectural decision rather than a cosmetic console setting.

2.2 Availability Zones

An Availability Zone (AZ) is one or more discrete data centers with independent power, networking, and physical facilities inside a Region. AZs are connected by high-bandwidth, low-latency links. Deploying application capacity across multiple AZs reduces dependence on a single facility-level failure domain.

Important: Multiple EC2 instances in one Availability Zone improve host redundancy but do not provide multi-AZ resilience. Failure-domain diversity requires placement across AZs.

2.3 Edge Infrastructure

Edge locations and related AWS edge infrastructure bring services such as content delivery, DNS resolution, security enforcement, and request acceleration closer to users. Edge infrastructure is not interchangeable with an Availability Zone: it serves different networking and delivery purposes.

2.4 Region Selection Framework

Factor	Question	Risk if ignored
Compliance and residency	Where may data be stored and processed?	Regulatory or contractual breach
Latency	Which Region is closest to users and connected systems?	Slow application response
Service availability	Are required services and features present?	Architecture redesign or launch delay
Cost	How do service and data-	Unexpected recurring expense

Factor	Question	Risk if ignored
	transfer prices compare?	
Resilience	Is a second Region needed for disaster recovery?	Unacceptable outage duration
Ecosystem	Are partners, connectivity, and support capabilities available?	Operational friction

2.5 Scope: Global, Regional, and Zonal Resources

Scope	Examples	Operational consequence
Global or account-wide	Some identity, DNS, CDN, and account constructs	Changes may affect many Regions or the whole account.
Regional	VPCs, many managed services, load balancers	Select the Region explicitly and replicate when required.
Availability Zone	EC2 placement, EBS attachment locality, subnets	Dependencies must be co-located or designed across AZs.

2.6 High Availability and Disaster Recovery

High availability keeps a service operating through expected component failures, often within one Region using multiple AZs. Disaster recovery restores or continues business service after a larger event, potentially using another Region. Recovery Time Objective (RTO) describes the maximum acceptable time to restore service. Recovery Point Objective (RPO) describes the maximum acceptable data loss measured in time.

DR pattern	Cost	Recovery speed	Typical approach
Backup and restore	Lowest	Slowest	Store backups and rebuild after an event.
Pilot light	Low to medium	Moderate	Keep core data services active; scale application tiers during recovery.
Warm standby	Medium to high	Fast	Run a smaller complete environment and scale it up.
Multi-site active/active	Highest	Fastest	Serve traffic from multiple locations continuously.

2.7 Multi-AZ Reference Pattern

```

Users
|
DNS / Edge
|
Regional Load Balancer
|-----|
Public subnet A      Public subnet B
|                   |
App subnet A        App subnet B
|-----|
Multi-AZ database / replicated data layer

```

2.8 Operational Checklist

- Confirm the active Region before every deployment and destructive action.
- Use at least two AZs for production services requiring high availability.
- Avoid creating state that can be recovered only from one AZ.
- Document RTO and RPO before selecting a disaster-recovery pattern.
- Test failover and recovery; architecture diagrams alone do not prove recoverability.

Chapter Summary

- Regions are geographic boundaries; Availability Zones are isolated failure domains within Regions.
- Multi-AZ design supports high availability, while multi-Region design may support disaster recovery or global service delivery.
- Region selection must include compliance, latency, service availability, cost, and resilience.

Interview Questions

1. What is the difference between a Region and an Availability Zone?
2. Why is deploying two instances in one AZ not sufficient for multi-AZ availability?
3. Explain RTO and RPO.
4. Compare backup-and-restore with warm standby.
5. What factors would you use to choose an AWS Region?
6. What is the role of edge infrastructure?

Hands-on Lab 2 - Region and Resilience Assessment

Objective: Compare two candidate Regions and create a multi-AZ deployment decision record.

Tasks

1. Choose a hypothetical application and identify its primary user geography.
2. Compare two candidate Regions for latency, required services, data residency, estimated cost, and disaster-recovery options.
3. List the Availability Zones visible to your account in the chosen Region.
4. Draw a two-AZ architecture with public and private subnets.

5. Specify an RTO and RPO and select a matching recovery pattern.

Validation

- The decision is based on explicit criteria.
- The architecture separates availability from disaster recovery.

CHAPTER 3

AWS Account Setup, Free Usage Controls, and Root Security

Build a safe learning account before creating infrastructure

Learning Objectives

- Secure the root user and configure MFA.
- Create an administrative access path without routine root use.
- Understand that Free Tier and trial offers have service-specific limits.
- Configure cost visibility and alerts.
- Apply account-level security and ownership controls.

3.1 Account as a Security Boundary

An AWS account is a billing, ownership, quota, and security boundary. Every resource belongs to an account. Account setup therefore establishes the foundation for identity, logging, cost controls, support, and incident response.

3.2 Root User

The root user is created from the account email address and has account-level capabilities that cannot be fully restricted by ordinary IAM policy. It should be used only for tasks that specifically require root credentials. Routine console work, CLI access, application access, and automation must use delegated identities.

Root security baseline: Use a unique email address, a unique high-entropy password, phishing-resistant MFA where practical, secure recovery methods, and monitored use. Do not create root access keys.

3.3 Initial Security Sequence

1. Enable MFA for the root user.
2. Confirm alternate security, operations, and billing contacts.
3. Create or enable a delegated administrative identity, preferably through AWS IAM Identity Center for human access.
4. Enable audit logging and centralized log retention as the environment grows.
5. Configure budgets, cost alerts, and anomaly visibility.
6. Sign out of root and test delegated administration.
7. Document the emergency root-access procedure.

3.4 Free Usage Is Not Unlimited

AWS Free Tier, credits, and service trials are not blanket protection from charges. Each offer has eligibility rules, time periods, Regions, quantities, and service-specific conditions. Data transfer, public IPv4 addresses, storage, snapshots, NAT gateways, load balancers, and idle resources can create charges even in a learning environment.

Cost rule: Treat every resource as chargeable unless the current AWS pricing page explicitly confirms otherwise for your account and usage.

3.5 Billing and Cost Controls

- Enable access to billing information only for approved identities.
- Create a small monthly budget with notification thresholds.
- Use cost allocation tags such as Project, Environment, Owner, and CostCenter.
- Review the billing dashboard after each lab that creates persistent resources.
- Delete unused EBS volumes, snapshots, Elastic IP addresses, load balancers, NAT gateways, databases, and log retention that is no longer needed.

3.6 Account Contacts and Recovery

Security and billing incidents often occur when the primary account owner is unavailable. Organizations should use role-based contact addresses rather than one employee's mailbox, protect the email account with MFA, maintain recovery procedures, and ensure payment and contact details remain current.

3.7 Multi-Account Direction

A single account is acceptable for initial learning, but enterprises normally separate production, non-production, security, logging, and shared services into multiple accounts governed through AWS Organizations. Account separation limits blast radius and improves ownership, billing, and policy control.

Chapter Summary

- The root user is an emergency account identity, not a daily administrator.
- Free usage controls require active budgets, alerts, and cleanup.
- Account contacts, delegated administration, logging, and recovery procedures should be established before production workloads.

Interview Questions

1. Why should the root user not be used for daily work?
2. What account-level actions should be taken immediately after registration?
3. Why can a Free Tier account still receive charges?
4. Why are multi-account environments common in enterprises?
5. What information should an emergency root-access procedure contain?

Hands-on Lab 3 - Secure the Learning Account

Objective: Apply a minimum account security and cost-control baseline.

Tasks

1. Enable MFA for the root user and verify that no root access key exists.
2. Configure delegated administrator access and sign in without root.
3. Run the identity check appropriate to the access method.
4. Create a monthly cost budget and at least two notification thresholds.
5. Add alternate security, operations, and billing contacts.
6. Create a written cleanup checklist for all future labs.

Validation

- Daily administration works without root.
- A budget and notification recipients are configured.
- Root credentials are stored securely and not used by CLI or applications.

CHAPTER 4

AWS Management Interfaces

Console, CLI, CloudShell, SDKs, APIs, profiles, and credential safety

Learning Objectives

- Choose between Console, CLI, CloudShell, SDK, and direct API use.
- Install and configure AWS CLI v2 safely.
- Use named profiles and verify caller identity.
- Understand credential provider chains and temporary credentials.
- Automate repeatable resource queries and changes.

4.1 One Platform, Multiple Interfaces

Interface	Best use	Primary risk
Management Console	Exploration, visual inspection, one-off administration	Manual drift and wrong Region/account
AWS CLI	Automation, scripting, repeatable operations	Broad credentials and destructive commands
CloudShell	Quick authenticated browser terminal	Assuming its environment is permanent
SDK	Application and automation integration	Credential handling and retry design
API	Underlying protocol and advanced integrations	Signing, pagination, throttling, and error handling

4.2 Console Discipline

- Confirm account alias or account ID and Region before making changes.
- Use tags and resource names that make ownership and environment obvious.
- Avoid console-only production configuration; capture repeatable state as code.
- Use separate browser profiles or clear visual markers for production and non-production accounts.

4.3 AWS CLI Installation and Verification

```
aws --version
aws sts get-caller-identity
aws configure list
aws configure list-profiles
```

The identity check returns the account and principal used for the request. Run it before administrative or destructive commands, especially when multiple profiles or roles are available.

4.4 Named Profiles

```
aws configure --profile learning-admin
aws sts get-caller-identity --profile learning-admin
aws ec2 describe-regions --profile learning-admin --output table
```

4.5 Output and Querying

```
aws ec2 describe-instances --region ap-south-1 --query 'Reservations[].Instances[].
[InstanceId,State.Name,InstanceType]' --output table
```

Server-side API filters and client-side query expressions reduce output and make scripts more deterministic. Scripts must still handle empty responses, pagination, retries, non-zero exit codes, and partial failures.

4.6 Credential Safety

- Prefer federated or role-based temporary credentials for people and workloads.
- Never embed credentials in source code, images, shell history, user data, or repositories.
- Do not copy credentials into chat messages or tickets.
- Use roles for EC2, containers, and Lambda rather than static access keys.
- Rotate or revoke exposed credentials immediately and review CloudTrail activity.

4.7 CloudShell

AWS CloudShell provides a browser-based shell with AWS CLI and tools available. It is useful for quick queries and learning. Its convenience does not eliminate the need to verify account, Region, and permissions, and important scripts should live in version control rather than only in the shell environment.

4.8 SDK Concepts

```
import boto3
from botocore.exceptions import ClientError

session = boto3.Session(profile_name="learning-admin", region_name="ap-south-1")
ec2 = session.client("ec2")

try:
    response = ec2.describe_instances()
    for reservation in response.get("Reservations", []):
        for instance in reservation.get("Instances", []):
            print(instance["InstanceId"], instance["State"]["Name"])
except ClientError as exc:
    print(f"AWS API error: {exc}")
    raise
```

4.9 Automation Quality Requirements

- Make operations idempotent where possible.
- Log the account, Region, resource identifiers, and requested action.
- Use dry-run or read-only discovery before destructive changes.
- Handle throttling with bounded exponential backoff.
- Use explicit timeouts and validate the final resource state.

- Require human approval for high-impact production actions.

Chapter Summary

- The Console is valuable for learning, but CLI, SDK, and Infrastructure as Code provide repeatability.
- Identity and Region verification are mandatory safety checks.
- Temporary credentials and workload roles are safer than long-lived static keys.

Interview Questions

1. When would you use CloudShell instead of a locally installed CLI?
2. What does `aws sts get-caller-identity` verify?
3. Why are named profiles useful?
4. What is a credential provider chain?
5. How should automation handle API throttling and partial failure?
6. Why is console-only production administration risky?

Hands-on Lab 4 - CLI Safety and Inventory

Objective: Create a safe CLI workflow for inventorying resources.

Tasks

1. Install or open AWS CLI v2 and verify its version.
2. Authenticate using a delegated identity and run `aws sts get-caller-identity`.
3. List configured profiles and confirm the default Region.
4. Use a query expression to list EC2 instance IDs, states, and types in a table.
5. Export a read-only inventory result to JSON.
6. Write a shell function that prints identity and Region before running another AWS command.

Validation

- No root credentials or hard-coded keys are used.
- The command output identifies the intended account and Region.

CHAPTER 5

Identity and Access Management Deep Dive

Authorization policies, roles, federation, evaluation logic, and enterprise access design

Learning Objectives

- Explain identities, policies, roles, trust policies, and sessions.
- Read and write least-privilege IAM policies.
- Apply policy evaluation logic, including explicit deny.
- Use roles and temporary credentials for workloads and cross-account access.
- Understand permission boundaries, service control policies, and federation.

5.1 Authentication and Authorization

Authentication proves the identity of a principal. Authorization determines whether that principal may perform a requested action on a resource under stated conditions. A successful login does not imply permission to every service.

5.2 IAM Building Blocks

Component	Purpose	Production guidance
User	Long-lived identity inside one account	Avoid for workforce at scale; prefer federation.
Group	Permission grouping for IAM users	Useful for simple accounts; groups cannot be assumed.
Role	Assumable identity with temporary credentials	Preferred for workloads and cross-account access.
Policy	JSON permission document	Use least privilege and reusable customer-managed policies.
Trust policy	Defines who may assume a role	Restrict principals and conditions.
Session	Temporary credentials for an assumed role	Short duration and auditable session identity.

5.3 Policy Anatomy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadProjectArtifacts",
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::example-project-artifacts/*",
      "Condition": {
        "Bool": {"aws:SecureTransport": "true"}
      }
    }
  ]
}
```

A statement combines effect, action, resource, and optional conditions. Wildcards simplify policies but expand the blast radius. Production policies should scope actions and resources as tightly as the service supports.

5.4 Evaluation Logic

1. A request is denied by default.
2. Applicable identity-based and resource-based permissions are evaluated.
3. An explicit allow can grant access when no higher-level restriction blocks it.
4. An explicit deny in any applicable policy overrides an allow.
5. Permission boundaries, session policies, service control policies, and resource policies may reduce the effective permission set.

Key rule: A permission boundary does not grant permission. It limits the maximum permissions that an identity-based policy may grant.

5.5 Roles and Trust

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Service": "ec2.amazonaws.com"},
    "Action": "sts:AssumeRole"
  }]
}
```

The trust policy permits the EC2 service to assume the role. A separate permissions policy defines what the resulting session may do. Trust and permission are separate questions: who may obtain the role, and what may the role do?

5.6 Workload Identity

Applications running on AWS should receive credentials from an attached role or service-specific workload identity mechanism. The platform supplies temporary credentials and rotates them. This avoids distributing static secrets across servers, containers, and deployment pipelines.

5.7 Cross-Account Access

A principal in one account can assume a role in another account when the target role trust policy and the source principal permissions both permit it. Cross-account roles support centralized workforce identity and reduce duplicate users. External IDs, organization conditions, session tags, and source identity can strengthen third-party and enterprise access patterns.

5.8 Federation and IAM Identity Center

Workforce users should usually authenticate through an organization identity provider and receive temporary access through AWS IAM Identity Center or another federation pattern. Centralized authentication improves lifecycle management, MFA enforcement, account access assignment, and auditability.

5.9 Guardrails

- Permission boundaries cap delegated IAM administration.
- Service control policies in AWS Organizations restrict the maximum permissions available in member accounts.
- Resource policies control access directly on supported resources.
- Conditions can require encryption, approved networks, MFA, tags, source accounts, or organization membership.
- Access Analyzer and policy simulation help discover unintended or missing access.

5.10 Least-Privilege Method

1. Define the job task and exact API operations.
2. Scope resources by ARN, account, Region, project, and environment where supported.
3. Add conditions for secure transport, source, tags, or session context.
4. Test denied and allowed scenarios.
5. Observe real activity and refine permissions.
6. Review periodically and remove unused access.

Chapter Summary

- IAM separates authentication, trust, and authorization.
- Roles and temporary sessions are the default production pattern for workforce and workload access.
- Effective permissions are the intersection of grants and guardrails, with explicit deny taking precedence.

Interview Questions

1. What is the difference between a role trust policy and a permissions policy?
2. Explain IAM policy evaluation and explicit deny.
3. What does a permission boundary do?
4. How does cross-account role assumption work?

5. Why are roles preferable to access keys for EC2 applications?
6. Compare an IAM policy, resource policy, and service control policy.
7. How would you implement least privilege for a deployment pipeline?

Hands-on Lab 5 - Least-Privilege Role Design

Objective: Create and test a narrowly scoped read-only role.

Tasks

1. Create a customer-managed policy that permits describing EC2 instances and reading selected CloudWatch metrics.
2. Create a role with an appropriate trust policy for your test principal.
3. Assume the role and verify the session identity.
4. Test an allowed read action.
5. Test a denied write action such as starting or terminating an instance.
6. Review the access in CloudTrail or the available audit view.
7. Remove the test role and policy after validation.

Validation

- Allowed operations succeed and write operations fail.
- The session uses temporary credentials.
- The policy contains no unnecessary wildcard administrative actions.

CHAPTER 6

Amazon EC2 Compute Engineering

Instances, AMIs, families, purchasing models, bootstrapping, access, and operations

Learning Objectives

- Explain the EC2 instance lifecycle and launch configuration.
- Select instance families based on CPU, memory, storage, network, and accelerator needs.
- Compare On-Demand, Savings Plans, Reserved capacity concepts, and Spot usage.
- Launch and secure Linux instances.
- Use user data, tags, roles, health checks, and replacement-oriented operations.

6.1 EC2 Operating Model

Amazon Elastic Compute Cloud provides resizable virtual compute capacity. An EC2 instance is created from an Amazon Machine Image (AMI), placed into a subnet, assigned compute characteristics by an instance type, connected to storage, and controlled by security groups and IAM roles. EC2 gives flexibility but also leaves the guest operating system and application lifecycle largely under customer control.

6.2 Launch Components

Component	Purpose	Production decision
AMI	Operating system and initial software image	Use approved, patched, reproducible images.
Instance type	vCPU, memory, network, storage, accelerator profile	Benchmark the actual workload.
Subnet/AZ	Network placement and failure domain	Distribute capacity across AZs.
Security group	Stateful traffic rules	Allow only required sources and ports.
Key pair or SSM	Administrative access method	Prefer managed, auditable access; protect private keys.
IAM role	Temporary AWS API permissions	Never store application access keys on disk.
EBS volumes	Persistent block storage	Encrypt, snapshot, monitor, and right-size.
User data	Bootstrapping script or cloud-init data	Make idempotent and avoid embedded secrets.
Tags	Ownership and automation metadata	Require Project, Environment, Owner, and cost tags.

6.3 Instance Families

Family pattern	Best fit	Examples of workload traits
General purpose	Balanced compute and memory	Web servers, development tools, small services
Compute optimized	High CPU demand	Batch processing, media encoding, high-performance web tiers
Memory optimized	Large working sets	In-memory caches, analytics, memory-intensive databases
Storage optimized	High local storage throughput or IOPS	Distributed data processing, search, data warehouses
Accelerated computing	GPU or specialized accelerators	Machine learning, rendering, scientific workloads

Instance names and generations evolve. Engineers should select by measured workload requirement, current-generation availability, architecture compatibility, and price-performance—not by memorizing one fixed list.

6.4 CPU Architectures

AWS offers x86 and Arm-based instance options. Arm instances can provide attractive price-performance, but binaries, agents, containers, libraries, and vendor software must support the architecture. Build pipelines should produce and test the correct artifacts or multi-architecture container images.

6.5 Purchasing Models

Model	Best use	Operational caution
On-Demand	Uncertain, short-lived, new, or spiky workloads	Highest flexibility; optimize after usage is understood.
Savings Plans / commitments	Steady eligible compute usage	Commitment risk if architecture or demand changes.
Reserved capacity concepts	Predictable long-running usage or capacity needs	Understand whether discount, capacity reservation, or both are required.
Spot	Fault-tolerant and interruption-ready workloads	Instances can be interrupted; design checkpointing and diversification.
Dedicated options	Compliance, licensing, or tenancy requirements	Higher cost and additional capacity planning.

6.6 Instance Lifecycle

- Pending: capacity is being prepared.
- Running: instance is active and billable according to applicable dimensions.
- Stopping/stopped: compute is not running, but attached storage and other resources may remain chargeable.
- Shutting-down/terminated: instance is being permanently removed; configured volumes may also be deleted.
- Rebooting: operating system restarts while the instance remains allocated.

Cost trap: Stopping an instance does not delete its EBS volumes, snapshots, Elastic IP or public IPv4 charges, load balancers, or other dependent resources.

6.7 Secure Administrative Access

For Linux, SSH key pairs can prove identity, but exposed port 22 and unmanaged private keys create risk. AWS Systems Manager Session Manager can provide auditable access without opening inbound SSH when the instance, IAM role, network path, and agent are configured. For Windows, use secure administrative access patterns and avoid broad public RDP exposure.

6.8 Security Groups

A security group is a stateful virtual firewall associated with network interfaces. Inbound and outbound rules specify allowed traffic. Return traffic for an allowed connection is automatically permitted. Rules should reference trusted CIDR ranges or other security groups and avoid unrestricted administrative ports.

```
# Example intent, not a universal production rule:
Inbound 443/tcp   Source: internet or approved edge/load-balancer security group
Inbound 22/tcp   Source: none when Session Manager is used
Outbound         Restricted to required destinations when operationally feasible
```

6.9 Bootstrapping with User Data

```
#!/bin/bash
set -euo pipefail

dnf -y update
dnf -y install nginx
systemctl enable --now nginx
printf '<h1>Healthy instance</h1>' > /usr/share/nginx/html/index.html
```

Production bootstrapping should be idempotent, observable, versioned, and independent of fragile external state. Retrieve secrets from a secrets service at runtime rather than writing them into user data, images, or repositories.

6.10 Immutable and Replaceable Operations

Manual server repair creates configuration drift. A stronger pattern builds an approved image, launches replacement instances from a launch template, validates health, shifts traffic, and terminates the old capacity. This makes rollback, scaling, and disaster recovery more predictable.

6.11 EC2 Troubleshooting

Symptom	Likely checks
Cannot connect	Instance state, route, public/private IP, security group, NACL, key or SSM configuration, OS firewall
Application unavailable	Process status, listener port, health endpoint, local firewall, security group, load balancer target health
High CPU	Application profile, runaway process, traffic spike, instance size, scaling policy
Disk full	Filesystem usage, logs, package cache, volume size, inode exhaustion, retention policy
Unexpected termination	Auto Scaling events, Spot interruption, operator action, health replacement, audit logs
Permission denied to AWS API	Instance role attachment, policy, trust, credential source, Region/resource ARN

Chapter Summary

- EC2 combines an AMI, instance type, network placement, storage, security groups, and workload identity.
- Instance family and purchasing choices should follow benchmarks and workload interruption tolerance.
- Production EC2 operations favor automated replacement, multi-AZ capacity, managed access, and role-based credentials.

Interview Questions

1. What information is required to launch an EC2 instance?
2. How do you choose an EC2 instance family?
3. Compare On-Demand and Spot usage.
4. Why does stopping an instance not necessarily stop all cost?
5. How are security groups stateful?
6. What are the benefits of Session Manager over public SSH?
7. What is an immutable-server deployment pattern?
8. How would you troubleshoot an unreachable EC2 application?

Hands-on Lab 6 - Secure Web Instance

Objective: Launch a Linux web server with controlled access and validation.

Tasks

1. Create a security group that permits HTTP only from your current public IP or an approved test source.
2. Launch a small Linux EC2 instance in a test subnet with an IAM role and encrypted root volume.
3. Use user data to install and start a web server.
4. Tag the instance with Project, Environment, Owner, and Expiry.
5. Connect using Session Manager where available; otherwise use a protected key and restricted SSH source.
6. Run curl locally and remotely to validate the web endpoint.
7. Inspect instance status checks and system logs.
8. Terminate the instance and verify that dependent resources are cleaned up.

Validation

- No unrestricted SSH or RDP rule exists.
- The instance uses an IAM role rather than stored access keys.
- The page is reachable only through the intended path.

Cleanup

- Terminate the instance.
- Delete the test security group after dependencies are removed.
- Check for remaining EBS volumes, snapshots, and public IP-related resources.

CHAPTER 7

Amazon Machine Images and Elastic Block Store

Persistent storage, snapshots, encryption, performance, backup, and image lifecycle

Learning Objectives

- Differentiate EBS, instance store, and object storage use cases.
- Select EBS volume types by IOPS, throughput, latency, durability, and cost.
- Create, attach, mount, resize, encrypt, snapshot, and restore EBS volumes.
- Build and govern AMIs.
- Design backup and recovery validation.

7.1 Block Storage Fundamentals

Block storage presents addressable storage devices to an operating system. Filesystems and databases use those blocks. Amazon EBS provides persistent block volumes for EC2. EBS volumes are created in an Availability Zone and attach to compatible instances in that AZ. Snapshots provide point-in-time incremental backups that can be used to create new volumes.

7.2 EBS versus Instance Store versus S3

Storage	Persistence	Interface	Best fit
EBS	Persists independently of an instance unless configured for deletion	Block device	Boot volumes, databases, filesystems
Instance store	Ephemeral relative to instance lifecycle and host events	Local block device	Caches, buffers, replicated temporary data
S3	Durable object storage independent of instances	Object API	Artifacts, backups, static assets, data lakes

7.3 EBS Volume Families

Type category	Workload pattern	Design note
General-purpose SSD	Boot volumes and broad transactional workloads	Balance cost with configurable performance.
Provisioned IOPS SSD	Latency-sensitive and high-IOPS databases	Provision and monitor IOPS and throughput deliberately.

Type category	Workload pattern	Design note
Throughput-optimized HDD	Large sequential workloads	Not suitable for boot volumes or small random I/O.
Cold HDD	Infrequently accessed sequential data	Low cost with lower performance.

Current volume names, limits, and performance ranges should be verified in official AWS documentation. The engineering method remains stable: measure I/O size, IOPS, throughput, queue depth, latency, burst behavior, and cost.

7.4 Attach and Mount Workflow

```
lsblk
sudo file -s /dev/nvme1n1
sudo mkfs -t xfs /dev/nvme1n1
sudo mkdir -p /data
sudo mount /dev/nvme1n1 /data
sudo blkid /dev/nvme1n1
# Add the UUID-based entry to /etc/fstab, then test:
sudo umount /data
sudo mount -a
findmnt /data
```

Safety: Formatting a device destroys existing filesystem data. Confirm the device, snapshot state, and mount requirement before running mkfs.

7.5 Resizing

Increasing EBS volume size does not automatically expand the partition or filesystem. After modifying the volume, inspect the device, extend the partition when one exists, and grow the filesystem using the correct operating-system tool. Validate both block-device and filesystem size afterward.

7.6 Snapshots

EBS snapshots are point-in-time, incremental backups. Crash-consistent snapshots capture the block state at a moment, but application-consistent recovery may require filesystem freeze, database flush, backup integration, or application quiescence. A snapshot is useful only when restoration is tested.

1. Define backup frequency from the RPO.
2. Define retention from business, legal, and cost requirements.
3. Encrypt snapshots and control KMS key access.
4. Copy or replicate backups when the recovery design requires another account or Region.
5. Automate lifecycle management.
6. Perform scheduled restore tests and measure RTO.

7.7 EBS Encryption

EBS encryption uses AWS KMS keys and protects data at rest, data moving between an instance and its attached EBS storage, and subsequent encrypted snapshots. Encryption should normally be enabled by default at the account and Region level, with key governance matching the workload's compliance and recovery requirements.

7.8 AMI Lifecycle

An AMI packages the information needed to launch instances, including root-volume snapshots and launch metadata. Production teams commonly use an image pipeline: start from a trusted base, patch, install agents and dependencies, run vulnerability and functional tests, publish an immutable version, deploy gradually, and retire obsolete images after dependent workloads are replaced.

AMI practice	Reason
Version every image	Supports traceability and rollback.
Do not embed secrets	Images are copied, shared, and inspected.
Patch and scan before release	Reduces exposure and drift.
Tag source commit and build ID	Connects runtime to build provenance.
Define deprecation and deletion policy	Avoids obsolete images and snapshot cost.

7.9 Performance Troubleshooting

Observation	Investigation
High latency	Volume type, provisioned performance, queue depth, instance EBS bandwidth, filesystem and application I/O
Low throughput	I/O size, sequential versus random pattern, throughput limits, instance limits
Burst depletion	Baseline versus burst model, sustained demand, move to provisioned performance
Filesystem read-only	Kernel logs, underlying I/O errors, filesystem health, safe recovery procedure
Volume not visible	AZ, attachment state, device mapping, NVMe naming, operating-system scan
Restore too slow	Snapshot initialization behavior, data access pattern, pre-warming strategy where applicable

Chapter Summary

- EBS provides persistent AZ-scoped block storage; snapshots provide incremental point-in-time backups.

- Volume selection requires workload measurement, not only capacity sizing.
- Encryption, automated snapshots, tested restores, and governed AMI pipelines are production requirements.

Interview Questions

1. Compare EBS, instance store, and S3.
2. Why must an EBS volume and attached instance normally be in the same AZ?
3. What is the difference between crash-consistent and application-consistent snapshots?
4. Why does increasing a volume not always increase filesystem space?
5. How does EBS encryption interact with snapshots?
6. Describe a secure AMI build pipeline.
7. How would you troubleshoot high EBS latency?

Hands-on Lab 7 - EBS Lifecycle and Recovery

Objective: Create, mount, snapshot, restore, resize, and validate an encrypted data volume.

Tasks

1. Launch or reuse a test EC2 instance in one Availability Zone.
2. Create an encrypted general-purpose SSD EBS volume in the same AZ.
3. Attach, format, and mount it at /data using a UUID-based fstab entry.
4. Create test files and record checksums.
5. Create a snapshot and tag it with purpose and expiry.
6. Create a replacement volume from the snapshot and attach it to the instance.
7. Mount the restored volume separately and validate file checksums.
8. Increase the original volume size and expand its filesystem safely.
9. Document the measured restore time.
10. Delete all test volumes and snapshots after validation.

Validation

- Data survives detach and reattach.
- Restored checksums match the source.
- Filesystem size reflects the enlarged volume.

Cleanup

- Unmount and detach test volumes.
- Delete volumes and snapshots.
- Terminate any temporary instance.

CHAPTER 8

Elastic Load Balancing and EC2 Auto Scaling

Health-based routing, horizontal scale, launch templates, policies, and safe deployments

Learning Objectives

- Explain load-balancer listeners, rules, target groups, health checks, and TLS termination.
- Choose among Application, Network, Gateway, and legacy load-balancer use cases.
- Configure launch templates and Auto Scaling groups.
- Use target tracking, step, scheduled, and predictive scaling concepts.
- Design stateless, multi-AZ, self-healing application tiers.

8.1 Why Load Balancing Exists

A single instance is a capacity limit and a failure point. Elastic Load Balancing distributes incoming traffic across healthy targets in one or more Availability Zones. The load balancer provides a stable service endpoint while application instances can be added, replaced, patched, or removed.

8.2 Core Components

Component	Function
Load balancer	Receives client connections and selects targets.
Listener	Accepts a protocol and port, such as HTTPS 443.
Rule	Routes requests based on priority and matching conditions.
Target group	Logical set of EC2 instances, IP addresses, containers, or functions depending on type.
Health check	Tests whether a target should receive traffic.
Security group	Controls allowed network paths for applicable load-balancer types.

8.3 Load Balancer Selection

Type	Layer / focus	Typical use
Application Load Balancer	HTTP/HTTPS, Layer 7	Host/path routing, web APIs, microservices
Network Load Balancer	TCP/UDP/TLS, Layer 4	Very high performance, static IP needs, non-HTTP protocols

Type	Layer / focus	Typical use
Gateway Load Balancer	Network appliance insertion	Firewalls and traffic inspection appliances
Classic Load Balancer	Previous generation	Maintain legacy systems; prefer current types for new designs

8.4 Health Checks

A health check must represent application readiness, not merely that a process exists. A useful endpoint verifies critical local dependencies without causing expensive downstream work. Thresholds should avoid both slow failure detection and unstable flapping.

Health design: Separate liveness, readiness, and deep diagnostic signals. A load balancer readiness check should answer whether the target can safely receive traffic now.

8.5 TLS and Traffic Paths

Application Load Balancers can terminate TLS using certificates. The target connection may use HTTP or HTTPS depending on security requirements. Security groups should enforce the intended path: clients reach the load balancer, and instances accept application traffic only from the load balancer security group.

8.6 Auto Scaling Groups

An Auto Scaling group maintains a collection of EC2 instances between minimum, desired, and maximum capacity. It launches instances from a launch template, monitors health, replaces unhealthy instances, and changes capacity based on policies or schedules. When integrated with a load balancer, new instances register and terminated instances deregister automatically.

8.7 Launch Templates

- AMI version
- Instance type or compatible requirements
- Security groups
- IAM instance profile
- Block-device mappings
- User data
- Tags and metadata options
- Purchase configuration where applicable

8.8 Scaling Policies

Policy	Behavior	Example
Target tracking	Maintains a metric near a target value	Average CPU near 50% or request count per target

Policy	Behavior	Example
Step scaling	Adds or removes capacity in steps based on alarm magnitude	Add more instances for severe queue backlog
Scheduled scaling	Changes capacity at known times	Increase before a daily business peak
Predictive scaling	Forecasts recurring demand patterns	Prepare capacity for regular cycles

8.9 Stateless Application Design

Auto Scaling is most effective when instances are disposable. User sessions, uploads, application state, and configuration should not exist only on one instance. Store state in durable shared services, load configuration securely at startup, and make every instance equivalent enough to receive any request.

8.10 Deployment Patterns

Pattern	Method	Risk control
Rolling	Replace a portion of capacity at a time	Capacity and health thresholds
Blue/green	Create a complete new environment and switch traffic	Fast rollback to old environment
Canary	Send a small percentage to the new version	Observe before broad exposure
Instance refresh	Replace ASG instances using a new launch template version	Minimum healthy percentage and checkpoints

8.11 Troubleshooting

Symptom	Checks
Targets unhealthy	Health path, port, app startup, security-group path, matcher codes, target type
502/503 errors	Backend availability, connection errors, timeout mismatch, no healthy targets
Scaling does not occur	Metric, alarm state, policy attachment, min/max constraints, cooldown or warmup
New instance fails repeatedly	AMI, user data, dependency access, IAM role, capacity or instance compatibility
Uneven traffic	Long-lived connections, slow targets, stickiness, cross-zone behavior, application latency
Scale-in disrupts requests	Deregistration delay, lifecycle hooks, connection

Symptom	Checks
	draining, termination policy

Chapter Summary

- Elastic Load Balancing routes traffic only to healthy registered targets and provides a stable endpoint.
- Auto Scaling groups maintain desired capacity and replace unhealthy instances.
- Stateless services, accurate health checks, launch-template versioning, and multi-AZ placement enable safe horizontal scaling.

Interview Questions

1. Compare Application and Network Load Balancers.
2. What is the relationship between a listener, rule, and target group?
3. How does an Auto Scaling group replace unhealthy instances?
4. What is target tracking scaling?
5. Why should application instances be stateless?
6. How would you restrict instances so they accept traffic only from the load balancer?
7. Compare rolling, blue/green, and canary deployments.
8. How would you troubleshoot targets that remain unhealthy?

Hands-on Lab 8 - Multi-AZ Scaled Web Tier

Objective: Deploy a two-AZ web tier behind an Application Load Balancer and Auto Scaling group.

Tasks

1. Create or select a VPC with two subnets in different Availability Zones.
2. Create one security group for the load balancer and one for application instances.
3. Allow HTTP or HTTPS to the load balancer from the intended client source.
4. Allow the application port to instances only from the load-balancer security group.
5. Create a launch template with an approved AMI, IAM role, encrypted storage, tags, and user data.
6. Create a target group with a health endpoint.
7. Create an Application Load Balancer across two subnets.
8. Create an Auto Scaling group with minimum 2, desired 2, and a safe maximum.
9. Attach a target-tracking policy.
10. Terminate one instance manually and observe automatic replacement.
11. Generate controlled traffic and inspect scaling and target health.
12. Delete the stack in dependency order.

Validation

- At least one healthy target exists in each AZ.
- Instances are not directly exposed on the application port.
- Replacement occurs after an instance is terminated.

Cleanup

- Delete Auto Scaling group and launch template versions.
- Delete load balancer, listener, and target group.
- Delete test security groups after dependencies are gone.

CHAPTER 9

Amazon VPC Networking Foundations

CIDR planning, subnets, routes, gateways, security groups, NACLs, DNS, and private connectivity

Learning Objectives

- Design IPv4 CIDR ranges and subnets without overlap.
- Explain route-table longest-prefix matching and local routing.
- Build public, private, and isolated subnet patterns.
- Compare internet gateways, NAT gateways, and VPC endpoints.
- Differentiate security groups and network ACLs.
- Troubleshoot end-to-end network paths.

9.1 VPC as a Network Boundary

Amazon Virtual Private Cloud provides a logically isolated virtual network. You select IP address ranges, create subnets, configure route tables and gateways, attach security controls, and place resources into the resulting network. A VPC belongs to one Region and spans its Availability Zones; each subnet belongs to one AZ.

9.2 CIDR Planning

CIDR defines an address range and prefix length. A smaller prefix number represents a larger network. Address planning should reserve space for growth, multiple AZs, environment separation, and connectivity to other VPCs or on-premises networks. Overlapping ranges complicate or prevent routing between networks.

Example	Approximate usable design intent
10.20.0.0/16	Large VPC address space
10.20.0.0/20	Public subnets and future expansion block
10.20.16.0/20	Private application subnets
10.20.32.0/20	Private data or isolated subnets

Planning rule: Do not choose CIDR ranges independently for every project. Maintain an organization-wide IP address plan to prevent overlap.

9.3 Subnets

A subnet becomes public or private because of its route-table path and resource addressing—not because of its name. A public subnet commonly has a default route to an internet gateway, and internet-facing resources require suitable public addressing. A private subnet lacks a direct inbound internet route. An isolated subnet has no route to the internet and may contain data services.

9.4 Route Tables

Every subnet is associated with one route table. A route contains a destination and a target. AWS routing uses the most specific matching route. The local route enables communication within the VPC CIDR. Default routes such as 0.0.0.0/0 direct traffic not matched by a more specific route.

```
Public subnet route table
10.20.0.0/16    local
0.0.0.0/0      internet-gateway

Private application subnet route table
10.20.0.0/16    local
0.0.0.0/0      NAT-gateway-in-public-subnet

Isolated data subnet route table
10.20.0.0/16    local
```

9.5 Internet Gateway

An internet gateway attaches to a VPC and can be the target of routes for internet traffic. A route alone is not sufficient: instances also require public addressing where applicable, security rules, operating-system listeners, and a valid return path.

9.6 NAT Gateway

A public NAT gateway placed in a public subnet allows resources in private subnets to initiate outbound IPv4 internet connections without accepting unsolicited inbound internet connections. Private subnet route tables point the default route to the NAT gateway, while the NAT gateway's public subnet routes to the internet gateway.

Availability and cost: For resilient multi-AZ designs, use an appropriate NAT path per AZ and route each private subnet locally. NAT gateways also have hourly and data-processing cost, so VPC endpoints and architecture choices can reduce both dependency and expense.

9.7 VPC Endpoints

VPC endpoints provide private connectivity to supported AWS services without requiring traffic to traverse an internet gateway or NAT gateway. Gateway endpoints and interface endpoints have different architectures and pricing. Endpoint policies and security groups should restrict access to intended resources and principals.

9.8 Security Groups versus Network ACLs

Property	Security group	Network ACL
Attachment	Network interface / resource	Subnet boundary
State	Stateful	Stateless
Rules	Allow rules	Allow and deny rules
Evaluation	All applicable rules collectively	Ordered rule processing
Primary use	Resource-level traffic control	Subnet-level guardrail or

Property	Security group	Network ACL
		specialized control

9.9 DNS and Addressing

VPC DNS settings, Route 53 Resolver, private hosted zones, and resource hostnames affect service discovery. Production systems should use stable DNS names rather than hard-coded IP addresses. Public IPv4 addresses and Elastic IP addresses should be assigned only when justified, monitored, and removed when unused.

9.10 Flow Logs and Reachability

VPC Flow Logs capture metadata about network flows at supported interfaces, subnets, or VPC scope. They help investigate accepted and rejected traffic but are not packet payload captures. Reachability analysis, route inspection, security-group references, NACL rules, DNS testing, and application listeners together support systematic troubleshooting.

9.11 Packet-Path Troubleshooting Method

1. Define source IP, source port, destination IP or DNS name, destination port, and protocol.
2. Resolve DNS and confirm the expected address.
3. Check the source route table and most-specific route.
4. Check gateway, endpoint, peering, VPN, or transit path state.
5. Check security-group egress and ingress rules.
6. Check NACL rules in both directions, including ephemeral ports.
7. Check destination listener and operating-system firewall.
8. Check return route and asymmetric-routing risk.
9. Use flow logs and application logs to validate the result.

Chapter Summary

- A VPC spans a Region; subnets are AZ-scoped.
- Public and private behavior is determined by addressing, routes, gateways, and security controls.
- Security groups are stateful resource controls; NACLs are stateless subnet controls.
- Network troubleshooting must trace the complete forward and return path.

Interview Questions

1. What makes a subnet public?
2. How does route-table longest-prefix matching work?
3. Why is a NAT gateway placed in a public subnet?
4. Compare security groups and network ACLs.
5. Why are overlapping CIDR ranges a problem?

6. What is the purpose of a VPC endpoint?
7. Why might one NAT gateway per AZ improve resilience?
8. How would you troubleshoot an EC2 instance in a private subnet that cannot reach a package repository?

Hands-on Lab 9 - Two-AZ VPC

Objective: Build a production-style VPC foundation with public, private, and isolated routing.

Tasks

1. Create a VPC with a non-overlapping private IPv4 CIDR.
2. Create two public, two private application, and two isolated data subnets across two Availability Zones.
3. Create separate route tables for public, private, and isolated tiers.
4. Attach an internet gateway and route public subnets to it.
5. Create the required NAT design for the lab budget and route private subnets through it.
6. Create a gateway or interface VPC endpoint for a supported service and restrict its policy.
7. Create security groups for a load balancer, application tier, and database tier using security-group references.
8. Enable VPC Flow Logs to an approved destination with controlled retention.
9. Launch a test instance in a private subnet and validate outbound access and private service access.
10. Delete chargeable networking resources after recording the architecture.

Validation

- Each subnet is associated with the intended route table.
- The private instance has no direct public inbound path.
- The isolated tier has no default internet route.
- Traffic follows least-privilege security-group relationships.

Cleanup

- Delete NAT gateways first and release associated addresses when no longer needed.
- Delete endpoints, test instances, flow-log destinations as appropriate, route resources, subnets, gateways, and VPC.

CHAPTER 10

AWS Cost Management, Governance, and Foundation Project

Budgets, tags, pricing decisions, rightsizing, operational ownership, and Part A capstone

Learning Objectives

- Identify major AWS cost drivers and pricing dimensions.
- Use budgets, cost allocation tags, Cost Explorer concepts, and anomaly detection.
- Apply rightsizing, scheduling, commitment, storage lifecycle, and data-transfer optimization.
- Create ownership and deletion controls.
- Integrate Chapters 1-9 into a secure, resilient, observable, and cost-governed foundation.

10.1 Cost Is an Architecture Property

AWS cost is created by architecture and operations: runtime hours or seconds, instance size, storage capacity and performance, requests, data processing, public IP usage, load balancers, NAT gateways, snapshots, logs, cross-AZ and internet data transfer, support plans, and managed-service consumption. Cost optimization begins during design and continues through measurement and ownership.

10.2 Cost Visibility Tools

Capability	Purpose
Billing dashboard	High-level account charges and service totals
AWS Budgets	Threshold-based alerts for cost, usage, or commitments
Cost Explorer	Historical analysis, filtering, grouping, and forecasting concepts
Cost allocation tags	Attribute shared spend to owner, project, environment, or cost center
Cost and Usage Report	Detailed billing data for advanced analysis
Anomaly detection	Identify unusual cost patterns
Pricing Calculator	Model estimated architecture cost before deployment

10.3 Tagging Standard

Tag	Example	Purpose
Project	payments-api	Business or technical grouping

Tag	Example	Purpose
Environment	dev / test / prod	Lifecycle and policy
Owner	platform-team	Operational accountability
CostCenter	CC-1042	Financial allocation
ManagedBy	terraform	Automation source
DataClassification	internal	Security and compliance
Expiry	2026-08-15	Automated cleanup for temporary resources

10.4 Optimization Levers

- Rightsize compute based on observed CPU, memory, network, and storage behavior.
- Scale horizontally and scale in when demand falls.
- Stop non-production resources outside working hours when safe.
- Use Spot for interruption-tolerant workloads.
- Adopt commitments only after stable usage is understood.
- Choose storage classes, volume types, snapshot retention, and log retention intentionally.
- Reduce unnecessary cross-AZ, NAT, and internet data transfer.
- Delete unattached volumes, stale snapshots, unused load balancers, and orphaned addresses.
- Prefer managed services when reduced operations justifies their price.
- Include engineering labor and risk in total cost of ownership.

10.5 Unit Economics

Total monthly cost alone does not show efficiency. Teams should track cost per useful business unit, such as cost per request, active user, build, deployment, gigabyte processed, or order. Unit cost reveals whether increased spend is caused by healthy growth or declining efficiency.

10.6 Governance Model

1. Assign every resource an owner and environment.
2. Use policy and automation to require key tags.
3. Establish budgets at account, project, and environment levels.
4. Review cost anomalies and untagged resources on a defined schedule.
5. Create approved service patterns and quotas.
6. Require expiry for temporary environments.
7. Measure optimization savings without weakening reliability or security.
8. Document exceptions and their expiration dates.

10.7 Part A Reference Architecture

```

Internet users
|
DNS / TLS
|
Application Load Balancer (public subnets, two AZs)
|
Auto Scaling Group (private application subnets, two AZs)
|
Private data/service layer (isolated subnets)

Supporting controls:
- Federated or delegated administration with MFA
- EC2 instance roles and least-privilege IAM
- Encrypted EBS volumes and tested snapshots
- VPC endpoints for supported private AWS service access
- CloudWatch metrics/logs and audit logging
- Budgets, anomaly alerts, ownership tags, and expiry tags

```

10.8 Failure and Recovery Review

Failure	Expected behavior	Evidence
One EC2 instance fails	Load balancer stops routing; Auto Scaling replaces it	Target health and scaling activity
One AZ app capacity fails	Traffic continues to healthy capacity in another AZ	Multi-AZ targets and capacity
Bad launch template version	New instances fail health; rollout pauses or rolls back	Deployment checkpoints and alarms
Lost data volume	Restore from encrypted snapshot	Restore test and checksum
Credential exposure	Credential revoked; role/policy reviewed; activity investigated	CloudTrail and incident record
Unexpected cost spike	Budget or anomaly alert triggers investigation	Cost alert and owner workflow

10.9 Troubleshooting Unexpected Cost

1. Confirm account and billing period.
2. Group cost by service, Region, linked account, usage type, and tags.
3. Compare the increase date with deployments and scaling events.
4. Inspect untagged and newly created resources.
5. Check data transfer, NAT processing, public IPv4, snapshots, logs, and idle managed resources.
6. Contain the cost safely without deleting evidence or production dependencies.
7. Identify the owner and root cause.
8. Implement prevention: budget, quota, policy, schedule, lifecycle, or architecture change.

Chapter Summary

- Cloud cost must be designed, measured, allocated, and governed.
- Tags, budgets, anomaly detection, and owner workflows create accountability.
- Part A combines identity, compute, storage, load balancing, scaling, networking, resilience, and cost controls into one operational foundation.

Interview Questions

1. What are common hidden AWS cost drivers?
2. How do budgets differ from cost analysis?
3. Why are cost allocation tags important?
4. What is unit economics in cloud operations?
5. When should an organization purchase a compute commitment?
6. How can NAT and data transfer influence architecture cost?
7. How would you investigate an unexpected bill increase?
8. Describe the security and reliability controls in the Part A reference architecture.

Part A Capstone Project - Production-Style AWS Foundation

Objective: Design, deploy, validate, document, and remove a secure two-AZ web platform using the principles from Chapters 1-10.

Tasks

1. Create an architecture decision record covering Region, two AZs, CIDR plan, RTO, RPO, service choices, and cost assumptions.
2. Configure delegated human access with MFA and least-privilege roles.
3. Create a tagged two-AZ VPC with public, private application, and isolated data subnets.
4. Configure route tables, internet access for the public tier, private outbound or endpoint access as justified, and least-privilege security groups.
5. Create an encrypted, tested AMI or launch template-based EC2 configuration with an instance role and no embedded credentials.
6. Deploy an Application Load Balancer and Auto Scaling group with at least two healthy instances across AZs.
7. Create a versioned health endpoint and test instance replacement.
8. Attach or create an encrypted EBS data volume, snapshot it, and perform a restore validation.
9. Enable the available metrics, logs, flow logs, and audit evidence needed to troubleshoot the platform.
10. Create budget thresholds, cost tags, and an expiry date.
11. Run failure tests: terminate one instance, block a health check in a controlled manner, and validate recovery.
12. Create an operations runbook covering deployment, rollback, access, common failures, backup restore, and cleanup.

13. Delete all project resources after evidence is captured unless this environment is intentionally retained.

Validation

- The application remains available after one instance termination.
- All instances are private and receive traffic only through the load balancer.
- No long-lived credential is stored on an instance.
- Storage is encrypted and restore evidence exists.
- Every resource has ownership and cost tags.
- The final bill-impact review shows no unintended persistent resources.

Cleanup

- Export architecture, commands, screenshots, metrics, and test results.
- Delete in dependency order and confirm that NAT gateways, load balancers, EBS volumes, snapshots, addresses, and log destinations are not unintentionally retained.

Part A Command Reference

```
# Identity and configuration
aws sts get-caller-identity
aws configure list
aws configure list-profiles

# EC2 inventory
aws ec2 describe-instances --query 'Reservations[].Instances[].[InstanceId,State.Name,InstanceType,Placement.AvailabilityZone]' --output table

# EBS inventory
aws ec2 describe-volumes --query 'Volumes[].[VolumeId,State,Size,VolumeType,Encrypted,AvailabilityZone]' --output table

# Load balancers and target health
aws elbv2 describe-load-balancers --output table
aws elbv2 describe-target-health --target-group-arn <target-group-arn>

# Auto Scaling groups
aws autoscaling describe-auto-scaling-groups --output json

# VPC inventory
aws ec2 describe-vpcs --output table
aws ec2 describe-subnets --output table
aws ec2 describe-route-tables --output json
aws ec2 describe-security-groups --output json
```

Part A Production Readiness Checklist

- Root user protected with MFA and not used routinely.
- Human access is delegated or federated; workload access uses roles.
- Account, Region, environment, and owner are visible before changes.
- Architecture spans at least two AZs when high availability is required.
- Instances are replaceable and launched from versioned templates or images.
- Administrative ports are not broadly exposed.
- Application instances accept traffic only from approved upstream security groups.
- EBS is encrypted; backup retention and restore testing are documented.
- Health checks represent readiness and scaling limits are bounded.
- CIDRs do not overlap with connected networks.
- Private subnets use justified NAT or endpoint paths; isolated tiers have no default internet route.
- Metrics, logs, flow records, and audit records support investigation.
- Budgets, anomaly workflows, tags, cleanup, and expiry controls exist.
- Runbooks cover deployment, rollback, access, incident response, restore, and cost containment.

Glossary

Term	Meaning
AMI	Amazon Machine Image used as a launch source for EC2 instances.
ARN	Amazon Resource Name, a standardized identifier for AWS resources.
Auto Scaling group	Logical EC2 group that maintains and scales

Term	Meaning
	capacity.
Availability Zone	Isolated infrastructure location within an AWS Region.
CIDR	Notation describing an IP address range and prefix length.
EBS	Persistent block storage service commonly attached to EC2.
Elasticity	Ability to add and remove capacity as demand changes.
IAM role	Assumable identity that supplies temporary credentials.
Internet gateway	VPC gateway that can route internet traffic.
NAT gateway	Managed translation service for initiated outbound connectivity.
RPO	Maximum acceptable data-loss window measured in time.
RTO	Maximum acceptable service-restoration time.
Security group	Stateful virtual firewall attached to supported resources.
Subnet	AZ-scoped IP range inside a VPC.
Target group	Set of destinations receiving traffic from a load balancer.
VPC	Logically isolated virtual network in an AWS Region.

Official Reference Sources

The following official AWS documentation families were used as authoritative references for service behavior. Always consult the current version before production implementation because features, quotas, prices, Regions, and recommendations evolve.

- AWS General Reference and AWS Well-Architected guidance
- AWS Identity and Access Management User Guide
- AWS Command Line Interface User Guide
- Amazon EC2 User Guide
- Amazon EBS User Guide
- Elastic Load Balancing User Guide
- Amazon EC2 Auto Scaling User Guide
- Amazon VPC User Guide
- AWS Billing and Cost Management documentation